

Code Explanation

Delta Live Tables Test Code Explanation

This is a test code written in Python using the unittest framework to test the functionality of Delta Live Tables (DLT) in Databricks. The code creates a Spark session, defines test data, and tests two DLT functions: `order\_silver` and `customer\_silver`.

Overview

The test code is designed to validate the correctness of two DLT functions by mocking the `dlt.read` function to return test DataFrames. It checks the output DataFrames for the expected columns, values, and row counts.

Step 1: Setting up the Test Environment

This step involves creating a Spark session, defining test data, and making the Spark session available globally for the module.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDeltaLiveTables")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()

    import builtins
    builtins.spark = cls.spark
```

Step 2: Defining Test Data and Creating DataFrames

This step defines test data for customers and orders, creates DataFrames from the test data using predefined schemas.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "East"),
    ("C002", "Jane Smith", "jane@example.com", "West"),
]

order_data = [
    ("O001", "Product A", 10.0, 2, datetime.date(2023, 1, 15), "C001"),
    ("O002", "Product B", 15.0, 1, datetime.date(2023, 1, 20), "C002"),
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])
```

```

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)
cls.order_df = cls.spark.createDataFrame(order_data, schema=order_schema)

```

Step 3: Importing the Module Under Test

This step imports the `delta\_live\_tables` module, which contains the DLT functions to be tested.

```

from src import delta_live_tables
cls.dlt_module = delta_live_tables

```

Step 4: Testing the `order\_silver` Function

This step tests the `order\_silver` function by mocking the `dlt.read` function to return the test order DataFrame. It checks if the `TotalAmount` column is added and calculated correctly.

```

@patch('src.delta_live_tables.dlt.read')
def test_order_silver(self, mock_dlt_read):
    mock_dlt_read.return_value = self.order_df
    result_df = self.dlt_module.order_silver()
    self.assertTrue("TotalAmount" in result_df.columns)
    result_rows = result_df.collect()
    self.assertEqual(result_rows[0]["TotalAmount"], 20.0)
    self.assertEqual(result_rows[1]["TotalAmount"], 15.0)

```

Step 5: Testing the `customer\_silver` Function

This step tests the `customer\_silver` function by mocking the `dlt.read` function to return the test customer DataFrame. It checks the row count and verifies the data.

```

@patch('src.delta_live_tables.dlt.read')
def test_customer_silver(self, mock_dlt_read):
    mock_dlt_read.return_value = self.customer_df
    result_df = self.dlt_module.customer_silver()
    self.assertEqual(result_df.count(), 2)
    result = result_df.filter("CustId = 'C001'").collect()
    self.assertEqual(len(result), 1)
    self.assertEqual(result[0]["Name"], "John Doe")

```

Step 6: Stopping the Spark Session

This step stops the Spark session after the tests are completed.

```
@classmethod  
def tearDownClass(cls):  
    cls.spark.stop()
```